C# Collections

`System.Collections.Generic`

A Complete Developer's Guide

Interfaces · Classes · Code Examples

Dr. Ali A. Ahmed

Dept. of Computer Science – Faculty of Science –Minia University

Why Collections?

The Problem

Storing 1000+ customer names in plain variables is impossible.
You need a container that can grow, shrink, search, and sort.
But one container doesn't fit every scenario.

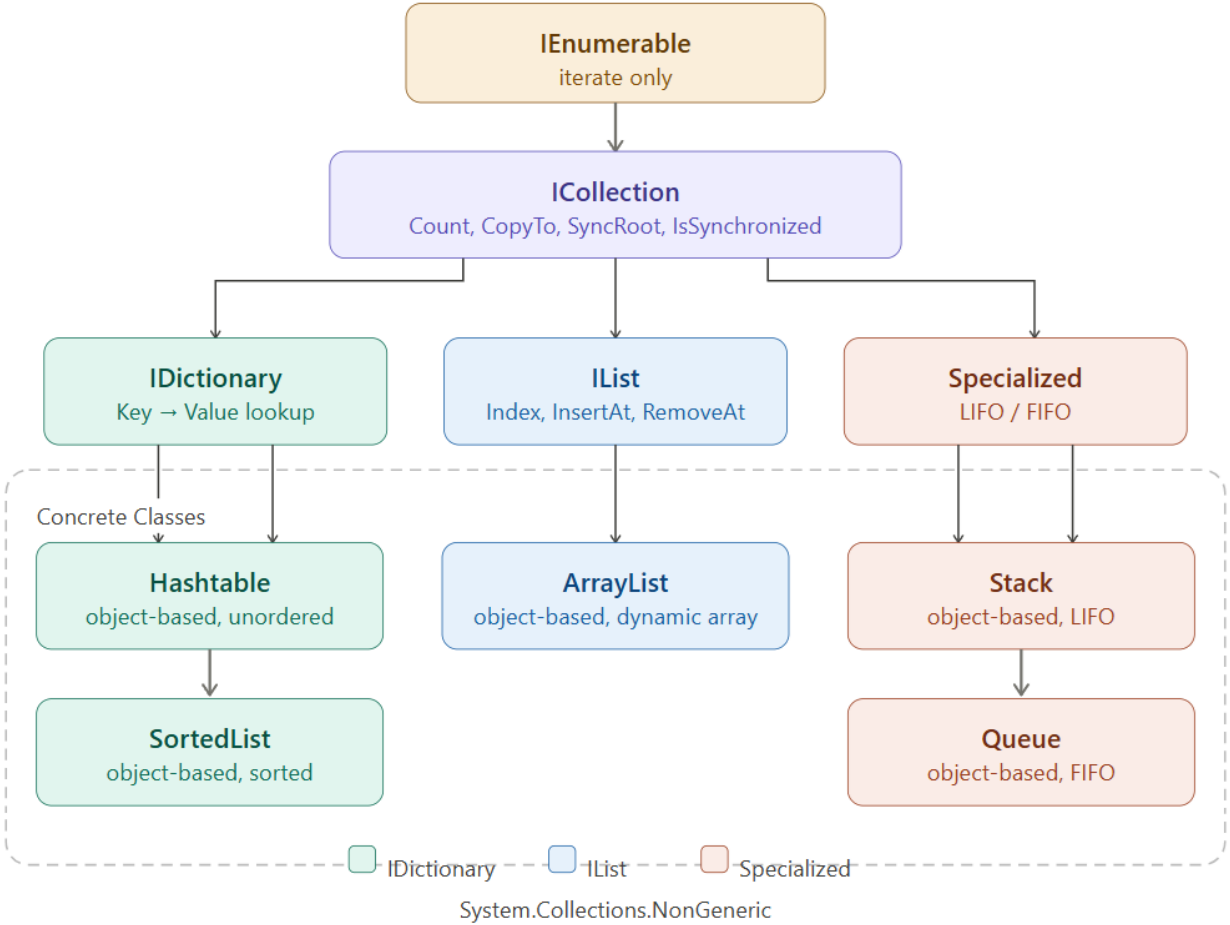
The Solution

- Choose the right collection for the job
- Understand interfaces (contracts)
- Know the performance trade-offs
- Write clean, type-safe code

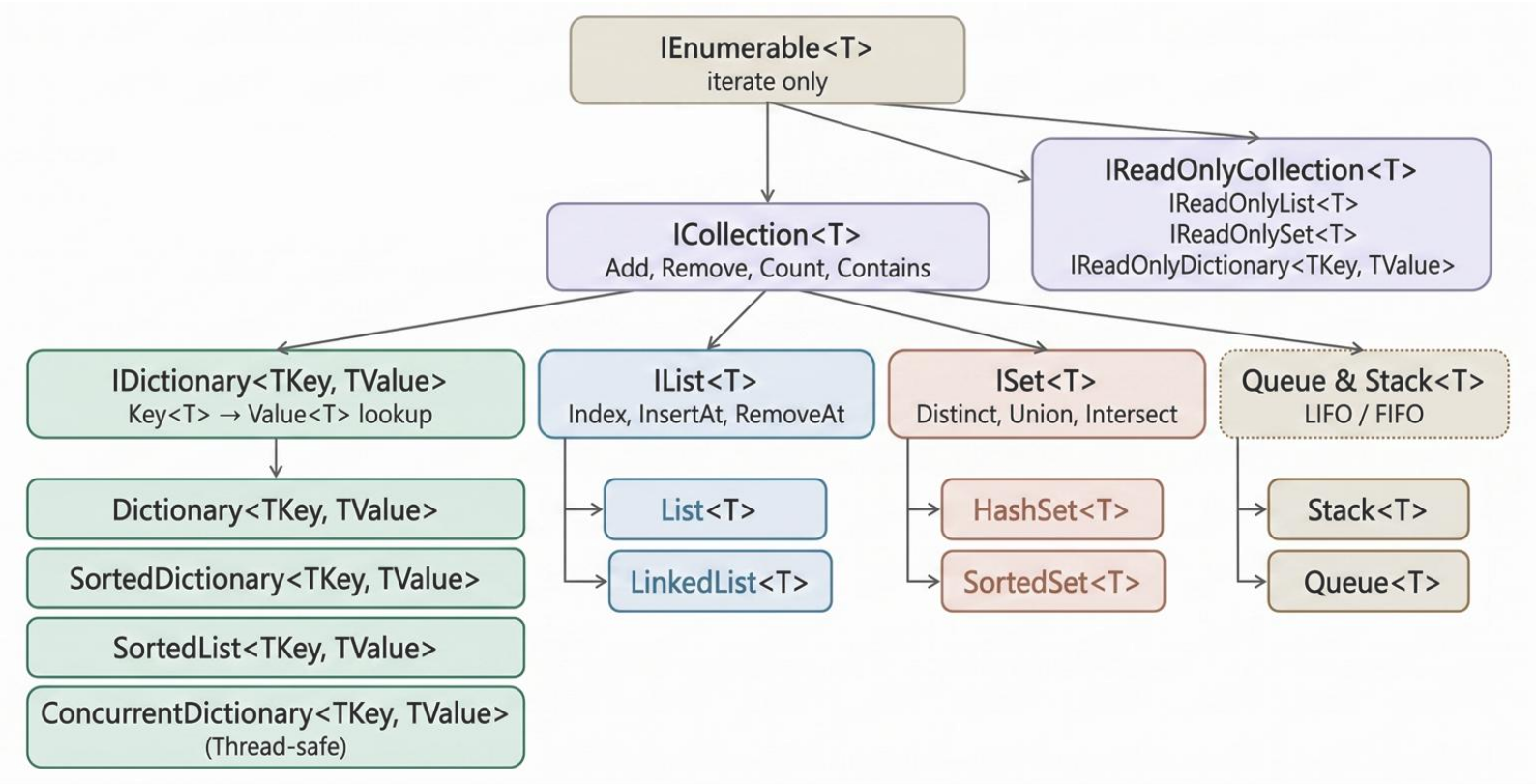
> *"The right collection isn't just the one that works — it's the one that works efficiently."*

Before .NET 2.0: non-generic collections used object — causing boxing/unboxing overhead & runtime type errors.
.NET 2.0+: System.Collections.Generic solved this with type-safe collections.

Non-Generic collections Hierarchy



Generic collections Hierarchy



IEnumerable<T> & ICollection<T>

IEnumerable<T>

The base of ALL collections.
Only capability: iterate with foreach.
No Add, Remove, or Count.

```
IEnumerable<int> nums = new List<int>{ 1,2,3 };

foreach (var n in nums)
    Console.WriteLine(n);

// nums.Add(4);   ❌ not available
```

ICollection<T>

Extends IEnumerable.
Adds: Add, Remove, Clear, Contains, Count, CopyTo.

```
ICollection<string> names = new List<string>();

names.Add("Ahmed");
names.Add("Sara");

Console.WriteLine(names.Count);     // 2
Console.WriteLine(names.Contains("Ahmed"));
// True
```

IList<T> · IDictionary<K,V> · ISet<T>

IList<T>

Extends ICollection.
Adds index [ ], Insert, RemoveAt, IndexOf.

```
IList<string> list = new
List<string>{ "A", "B", "C" };

Console.WriteLine(list[1]);    // B

list.Insert(1, "X");
// A, X, B, C

Console.WriteLine(
  list.IndexOf("B")); // 2
```

IDictionary<K,V>

Key→Value pairs.
Add, Remove, ContainsKey, TryGetValue.

```
IDictionary<string,int>  ages = new
Dictionary<string,int>();

ages["Ahmed"] = 30;
ages["Sara"]  = 25;

if (ages.TryGetValue("Ali", out int
a))
  Console.WriteLine(a);
else
  Console.WriteLine("Not found");
```

ISet<T>

No duplicates.
Union, Intersect, ExceptWith, IsSubsetOf.

```
ISet<int> a = new HashSet<int>
{1,2,3};

ISet<int> b = new HashSet<int>
{2,3,4};

a.IntersectWith(b);
// a = { 2, 3 }

a.UnionWith(b);
// a = { 2, 3, 4 }
```

Read-Only Interfaces (.NET 4.5+)

`IReadOnlyCollection<T>`	Count + iterate No Add/Remove	`IReadOnlyCollection<int> c = new List<int>{1,2,3};` `Console.WriteLine(c.Count); // 3`
`IReadOnlyList<T>`	Count + iterate + [ ] indexer Covariant in T	`IReadOnlyList<string> r = new List<string>{"A","B"};` `Console.WriteLine(r[0]); // A  – r.Add()` ❌
`IReadOnlyDictionary<K,V>`	Keys, Values, ContainsKey TryGetValue	`IReadOnlyDictionary<string,int> d = dict;` `Console.WriteLine(d["Ahmed"]); // read only`

⚠ Important Caveat

IReadOnly* interfaces hide mutation methods from the caller — they do NOT make the underlying object immutable.
If another reference to the original List<T> exists, it can still be modified.

Concrete Classes — Lists

List<T>

✓ Use when: Flexible, grow/shrink, index access

```
var students = new
List<string>();
students.Add("Ahmed");
students.Add("Sara");
students.Add("Omar");

students.Remove("Sara");
Console.WriteLine(students[0]);
// Ahmed

var found = students.
FindAll(s => s.StartsWith("A"));
// [ "Ahmed" ]
```

T[] Array

✓ Use when: Fixed size known upfront, max speed

```
int[] grades =
  new int[5]{ 90,85,78,92,88 };

Console.WriteLine(
    grades[2]); // 78

// grades.Add(95); ❌

Array.Sort(grades);
Console.WriteLine(
  string.Join(", ", grades));
// 78, 85, 88, 90, 92
```

LinkedList<T>

✓ Use when: Frequent insert/delete in middle

```
var playlist =
  new LinkedList<string>();
playlist.AddLast("Song A");
playlist.AddLast("Song C");
playlist.AddFirst("Song X");

var nodeC =
  playlist.Find("Song C");
playlist.AddBefore(
  nodeC, "Song B");
// X → A → B → C
```

Concrete Classes — Dictionaries

Dictionary<K,V>

✓ Default fast lookup

O(1) average

```
var sal = new
Dictionary<string,decimal>();
sal["Ahmed"] = 5000;
sal["Sara"]  = 7000;

Console.WriteLine(
  sal["Sara"]); // 7000

if (sal.TryGetValue("Ali", out
decimal v))
  Console.WriteLine(v);
else
  Console.WriteLine(“Not found");
```

SortedDictionary<K,V>

✓ Need keys always sorted

O(log n) — Red-Black Tree

```
var sd = new
SortedDictionary<string,int>();
sd["Zara"]  = 3;
sd["Ahmed"] = 1;
sd["Sara"]  = 2;

foreach (var kv in sd)
  Console.WriteLine(kv.Key);
// Ahmed, Sara, Zara
// (always sorted)
```

SortedList<K,V>

✓ Sorted + index access needed

O(log n) — Sorted Array

```
var sl = new
SortedList<string,int>();
sl["Zara"]  = 3;
sl["Ahmed"] = 1;

// Bonus: index access
Console.WriteLine(sl.Keys[0]);
//Ahmed
Console.WriteLine(sl.Values[0]);
// 1

// More memory-efficient
// than SortedDictionary
// for reads
```

Sets · Queue · Stack

HashSet<T> / SortedSet<T>

No duplicates · Set operations
HashSet O(1) · SortedSet O(log n)

```
var visited =  new HashSet<string>();
visited.Add("/home");
visited.Add("/about");
visited.Add("/home"); //dup!
Console.WriteLine(visited.Count);//2

var a = new HashSet<int>
  { 1, 2, 3, 4 };
var b = new HashSet<int>
  { 3, 4, 5, 6 };
a.IntersectWith(b);
// a = { 3, 4 }
```

Queue<T> – FIFO

First In, First Out
Enqueue · Dequeue · Peek

```
var q = new Queue<string>();
q.Enqueue("Ticket #1");
q.Enqueue("Ticket #2");
q.Enqueue("Ticket #3");

var next = q.Dequeue();
Console.WriteLine(next);
// Ticket #1

Console.WriteLine(q.Peek());
// Ticket #2
// (not removed)
```

Stack<T> – LIFO

Last In, First Out
Push · Pop · Peek

```
var undo = new Stack<string>();
undo.Push("Write 'Hello'");
undo.Push("Write ' World'");
undo.Push("Delete last word");

var last = undo.Pop();
Console.WriteLine(last);
// Delete last word

Console.WriteLine(undo.Peek());
// Write ' World'
```

The Golden Decision Guide

Need a flexible list with index access?	`List<T>`
Fixed size, maximum speed?	`T[]  Array`
Frequent insert/delete in the middle?	`LinkedList<T>`
Fast lookup by key?	`Dictionary<K,V>`
Keys must be sorted?	`SortedDictionary<K,V>`
Sorted + index access on pairs?	`SortedList<K,V>`
Unique elements, set operations?	`HashSet<T>`
Unique elements + sorted?	`SortedSet<T>`
Process items in arrival order? (FIFO)	`Queue<T>`
Undo/redo, depth-first? (LIFO)	`Stack<T>`
Multi-threaded / async environment?	`Concurrent*`
Expose API without allowing mutation?	`IReadOnly* interface`

Key Takeaways

- Interfaces define contracts — always program to the interface
- IEnumerable → ICollection → IList/IDictionary/ISet — understand the hierarchy
- IReadOnly* interfaces protect your API without copying data
- Choose your collection by what operations you need most
- Use Concurrent* collections for any multi-threaded scenario
- The right collection isn't just correct — it's efficient

`System.Collections.Generic`